

BÀI 7: KIỂM THỬ DỰA TRÊN ĐỘ ĐO DÒNG ĐIỀU KHIỂN

Mục tiêu:

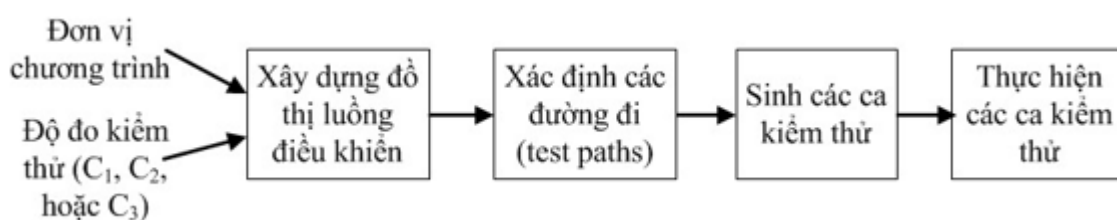
- **Về kiến thức:** cung cấp cho sinh viên kiến thức tổng quan về kiểm thử hộp trắng, kiến thức về kỹ thuật kiểm thử dòng điều khiển: phân biệt các độ đo kiểm thử, đồ thị dòng điều khiển ứng với mỗi độ đo
- **Về kỹ năng:** giúp sinh viên vẽ được đồ thị dòng điều khiển thoả mãn các độ đo cụ thể, thiết kế hệ thống testcase thoả mãn các độ đo kiểm thử

7.1 Kiểm thử dựa trên độ đo

Kiểm thử dựa trên độ đo là phương pháp chạy mã nguồn sao cho bao phủ một độ đo nào đó. Hình 7.1 mô tả quy trình kiểm thử dựa trên độ đo cho các đơn vị chương trình. Với mỗi đơn vị chương trình, đồ thị dòng điều khiển ứng với các độ đo C1 và C2 là giống nhau trong khi chúng khác với đồ thị dòng điều khiển ứng với độ đo C3. Với mỗi đơn vị chương trình và mỗi độ đo kiểm thử, chúng ta tiến hành xây dựng đồ thị dòng điều khiển tương ứng. Các đường đi của chương trình (xuất phát từ điểm bắt đầu, đi qua các đỉnh của đồ thị và kết thúc ở điểm cuối) được xác định sao cho khi chúng được thực hiện thì độ đo kiểm thử tương ứng được thoả mãn. Dựa trên ý tưởng của T. J. McCabe, số đường đi chương trình ứng với đồ thị dòng điều khiển của nó được tính bằng một trong các phương pháp sau:

- Số cạnh – số đỉnh + 2
- Số đỉnh quyết định + 1

Sau khi có được các đường đi của đơn vị chương trình cần kiểm thử, với mỗi đường đi, chúng ta sẽ sinh một ca kiểm thử tương ứng. Cuối cùng, các ca kiểm thử được thực hiện trên đơn vị chương trình nhằm phát hiện các lỗi.



Hình 7.1: Quy trình kiểm thử đơn vị chương trình dựa trên độ đo

7.1.1 Kiểm thử cho độ đo C1

Xét lại hàm foo có mã nguồn như Hình 7.2. Chúng ta xây dựng đồ thị dòng điều khiển ứng với độ phủ C1 cho hàm này như Hình 7.2. Để đạt được 100% độ phủ của độ đo C1, ta chỉ cần hai đường đi như sau để đảm bảo được tất cả các câu lệnh của hàm foo được kiểm thử ít nhất một lần. Để kiểm tra việc đảm bảo độ đo C1, chúng ta cần kiểm tra tất cả các lệnh/khối lệnh (1-8) đều được xuất hiện ít nhất một lần trong các đường đi này. Rõ ràng, hai đường đi này thỏa mãn điều kiện trên nên chúng ta đạt được 100% độ phủ C1.

1. 1; 2; 4; 5; 6; 7,8

2. 1; 2; 3

Với đường đi 1; 2; 4; 5; 6; 7,8, ta sẽ sinh một ca kiểm thử để nó được thực thi khi thực hiện ca kiểm thử này. Ý tưởng của việc sinh ca kiểm thử này là tìm một bộ giá trị đầu vào cho a, b, c và d sao cho điều kiện ứng với điểm quyết định 2 ($a == 0$) là sai và điều kiện ứng với điểm quyết định 5 ($(a == b) \parallel (c == d)$) là đúng. Giá trị đầu ra mong đợi (EO) của ca kiểm thử này là 1. Tương tự, ta sẽ sinh ca kiểm thử ứng với đường đi 1; 2; 3 với đầu ra mong đợi là 0. Chúng ta sẽ tìm một bộ đầu vào sao cho điều kiện ($a == 0$) là đúng. Bảng 7.1 là một ví dụ về hai ca kiểm thử được sinh ra bằng ý tưởng trên.

Bảng 7.1: Các ca kiểm thử cho độ đo C1 của hàm foo

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5; 6; 7,8	2, 2, 3, 5	1		
tc2	1; 2; 3	0, 3, 2, 7	0		

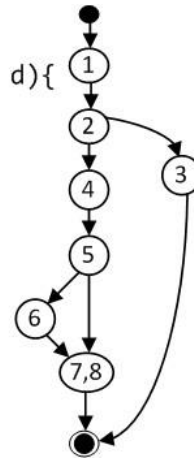
7.1.2 Kiểm thử cho độ đo C2

Như chúng ta đã biết, với mỗi đơn vị chương trình, đồ thị dòng điều khiển ứng với các độ đo C1 và C2 là giống nhau. Vì vậy, đồ thị dòng điều khiển ứng với độ đo C1 của hàm foo được mô tả ở Hình 7.2 cũng là đồ thị dòng điều khiển của hàm này ứng với độ đo C2. Tuy nhiên, để được 100% độ phủ của độ đo C2 chúng ta cần tối thiểu ba đường đi. Tại sao chúng ta biết được điều này? Như đã trình bày ở bài 6, chúng ta có hai cách để tính được con số này.

```

float foo(int a, int b, int c, int d){
1.  float e;
2.  if (a==0)
3.      return 0;
4.  int x = 0;
5.  if ((a==b) || (c==d))
6.      x = 1;
7.  e = 1/x;
8.  return e;
}

```



Hình 7.2: Hàm foo và đồ thị dòng điều khiển của nó.

Ví dụ, đồ thị dòng điều khiển của hàm foo có hai điểm quyết định là 2 và 5 nên chúng ta cần $2 + 1 = 3$ đường đi để đạt được 100% độ phủ của độ đo C2. Các đường đi cần thiết được liệt kê như sau. Rõ ràng với ba đường đi này, cả hai nhánh đúng và sai của hai điểm quyết 2 và 5 đều được kiểm tra.

1. 1; 2; 4; 5; 6; 7,8
2. 1; 2; 3
3. 1; 2; 4; 5; 7,8

Để sinh các ca kiểm thử ứng với các đường đi trên, chúng ta chỉ cần quan tâm đến đường đi (3) vì việc sinh các ca kiểm thử cho các đường đi (1) và (2) đã được trình bày ở mục kiểm thử cho độ đo C1. Với đường đi (3), ta chỉ cần chọn một bộ đầu vào sao cho điều kiện ứng với điểm quyết định 2 ($a == 0$) là sai và điều kiện ứng với điểm quyết định 5 ($(a == b) \parallel (c == d)$) cũng là sai. Giá trị đầu ra mong đợi của đường đi này là lỗi chia cho 0. Bảng 4.6 là một ví dụ về ba ca kiểm thử được sinh ra bằng ý tưởng trên ứng với các đường đi (1), (2), và (3).

Bảng 7.2: Các ca kiểm thử cho độ đo C2 của hàm foo

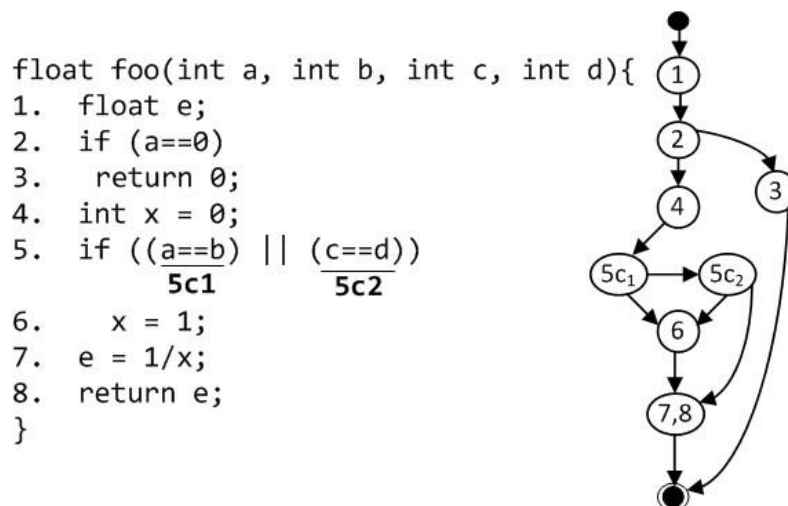
ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5; 6; 7,8	2, 2, 3, 5	1		
tc2	1; 2; 3	0, 3, 2, 7	0		
tc3	1; 2; 4; 5; 7,8	2,3,4,5	lỗi chia cho 0		

Độ đo C1 đảm bảo các câu lệnh được “viếng thăm” ít nhất một lần khi thực hiện tất cả các ca kiểm thử được sinh ra ứng với độ đo này. Đây là độ đo khá tốt và việc đảm bảo độ đo này trong thực tế cũng khá tốn kém. Tuy nhiên, qua ví dụ trên, chúng ta thấy rằng nếu chỉ sử dụng độ đo C1 với hai ca kiểm thử như trong bảng 7.2, lỗi chia cho

không sẽ không được phát hiện. Chỉ khi kiểm tra cả hai nhánh đúng sai của tất cả các điểm quyết định (các lệnh điều khiển) thì lỗi này mới được phát hiện như ca kiểm thử tc3 trong bảng 7.3.

7.1.3 Kiểm thử cho độ đo C3

Như đã trình bày ở phía trên, ứng với mỗi đơn vị chương trình, đồ thị dòng điều khiển ứng với độ đo C3 khác với đồ thị dòng điều khiển ứng với các độ đo C1 và C2. Ví dụ, đồ thị dòng điều khiển của hàm foo ứng với độ đo C3 được xây dựng như Hình 7.3. Với câu lệnh điều kiện 5, vì đây là điều kiện phức tạp nên ta phải tách thành hai điều kiện con cơ bản là $(a == b)$ và $(c == d)$ ứng với hai điểm quyết định 5c1 và 5c2 trong đồ thị dòng điều khiển. Từ câu lệnh 4, nếu điều kiện con $(a == b)$ đúng, ta không cần kiểm tra điều kiện con còn lại (vì điều kiện phức tạp là hoặc của hai điều kiện con cơ bản) và thực hiện câu lệnh 6. Nếu điều kiện con $(a == b)$ là sai, ta cần tiến hành kiểm tra điều kiện con cơ bản còn lại $(c == d)$. Nếu điều kiện này đúng, ta tiến hành câu lệnh 6. Ngược lại, chúng ta thực hiện các câu lệnh 7 và 8. Trong đồ thị này, chúng ta gộp hai lệnh 7 và 8 trong một đỉnh (đỉnh (7,8)) vì đây là hai câu lệnh tuần tự. Mục đích của việc này là nhằm tối thiểu số đỉnh của đồ thị dòng điều khiển. Một đồ thị có số đỉnh càng nhỏ thì chúng ta càng dễ dàng trong việc sinh các đường đi của chương trình và tránh các sai sót trong quá trình này.



Hình 7.3: Hàm foo và đồ thị dòng điều khiển của nó ứng với độ đo C3

Đồ thị dòng điều khiển của hàm foo ứng với độ đo C3 như Hình 7.3 có ba điểm quyết định là 2, 5c1 và 5c2 nên chúng ta cần $3 + 1 = 4$ đường đi để được 100% độ phủ của độ đo C3. Các đường đi cần thiết được liệt kê như sau:

1. 1; 2; 4; 5c1; 6; 7,8
2. 1; 2; 4; 5c1; 5c2; 6; 7,8

3. 1; 2; 4; 5c1; 5c2; 7,8

4. 1; 2; 3

Tương tự như các phương pháp kiểm thử độ đo C1 và C2, chúng ta dễ dàng sinh các ca kiểm thử tương ứng cho các đường đi chương trình như đã mô tả trên. Bảng 7.3 là một ví dụ về các ca kiểm thử cho hàm foo ứng với độ đo C3.

Bảng 7.3: Các ca kiểm thử cho độ đo C3 của hàm foo

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5c1; 6; 7,8	0, 2, 3, 5	0		
tc2	1; 2; 4; 5c1; 5c2; 6; 7,8	2, 2, 2, 7	1		
tc3	1; 2; 4; 5c1; 5c2; 7,8	2,3,4,5	lỗi chia cho 0		
tc4	1; 2; 3	2,3,4,4	1		

7.1.4 Kiểm thử vòng lặp

Cho dù chúng ta tiến hành kiểm thử các đơn vị chương trình với độ đo C3 (độ đo với yêu cầu cao nhất), phương pháp kiểm thử dòng điều khiển không thể kiểm thử các vòng lặp xuất hiện trong các đơn vị chương trình. Lý do là các đường đi sinh ra từ đồ thị dòng điều khiển không chứa các vòng lặp. Trong thực tế, lỗi hay xảy ra ở các vòng lặp. Vì lý do này, chúng ta cần sinh thêm các ca kiểm thử cho các vòng lặp nhằm giảm tỷ lệ lỗi của các đơn vị chương trình. Với mỗi đơn vị chương trình có vòng lặp, chúng ta cần quan tâm đến ba trường hợp sau:

- **Lệnh lặp đơn giản:** đơn vị chương trình chỉ chứa đúng một vòng lặp (thân của vòng lặp không chứa các vòng lặp khác).
- **Lệnh lặp liên kề:** đơn vị chương trình chỉ chứa các lệnh lặp kế tiếp nhau.
- **Lệnh lặp lồng nhau:** đơn vị chương trình chỉ chứa các vòng lặp chứa các lệnh lặp khác.

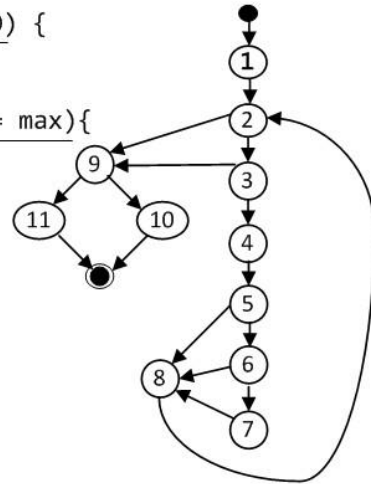
Để kiểm thử các đơn vị chương trình chỉ có lệnh lặp đơn giản, ta xét hàm average với mã nguồn đồ thị dòng điều khiển tương ứng với độ đo C3 như Hình 7.4.

Đồ thị dòng điều khiển của hàm average ứng với độ đo C3 như Hình 7.4 có năm điểm quyết định là 2, 3, 5, 6 và 9 nên chúng ta cần $5 + 1 = 6$ đường đi để được 100% độ phủ của độ đo C3.

```

double average(double value[], double min, double
max, int& tcnt, int& vcnt) {
double sum = 0;
int i = 1;
tcnt = vcnt = 0;
while (value[i] <> -999 && tcnt <100) {
    tcnt++;
    if (min<=value[i] && value[i]<= max){
        sum += value[i];
        vcnt ++;
    }
    i++;
} //end while
if (vcnt > 0)
    return sum/vcnt;
return -999;
} //end

```



Hình 7.4: Hàm average và đồ thị dòng điều khiển của nó ứng với độ đo C3.

Các đường đi cần thiết được liệt kê như sau:

1. 1; 2; 9; 10
2. 1; 2; 9; 11
3. 1; 2; 3; 9; 10
4. 1; 2; 3; 4; 5; 8; 2; 9; 11
5. 1; 2; 3; 4; 5; 6; 8; 2; 9; 11
6. 1; 2; 3; 4; 5; 6; 7; 8; 2; 9; 10

Bảng 7.4: Các ca kiểm thử cho độ đo C_3 của hàm average

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 9; 10				tc6
tc2	1; 2; 9; 11	[-999, ...], 1, 2	-999		
tc3	1; 2; 3; 9; 10				tc6
tc4	1; 2; 3; 4; 5; 8; 2; 9; 11	[0,-999], 1, 2	-999		
tc5	1; 2; 3; 4; 5; 6; 8; 2; 9; 11	[3,-999], 1, 2	-999		
tc6	1; 2; 3; 4; 5; 6; 7; 8; 2; 9; 10	[1,-999], 1, 2	1		

Với mỗi đường đi, chúng ta sẽ sinh một ca kiểm thử tương ứng. Bảng 7.4 mô tả các ca kiểm thử cho hàm average ứng với độ đo C3. Với mỗi ca kiểm thử, bộ dữ liệu đầu vào (inputs) gồm ba thành phần: double value[], double min, và double max. Với đường đi 1; 2; 9; 10, chúng ta không thể tìm được bộ dữ liệu đầu vào để đường đi này được thực thi. Thực vậy, điều kiện để thực thi đường đi này là value[0] = -999 (tức là điều kiện 2 là sai) và điều kiện 9 đúng (tức là vcnt > 0). Điều này không bao giờ xảy ra vì

nếu $\text{value}[0] = -999$ thì $\text{vcnt} = 0$. Tương tự, chúng ta cũng không thể sinh bộ kiểm thử với đường đi 1; 2; 3; 9; 10. Trong các trường hợp này, chúng ta không cần sinh các ca kiểm thử cho những đường đi này. Chúng sẽ được kiểm thử bởi các đường đi khác (Ví dụ: đường đi 1; 2; 3; 4; 5; 6; 7; 8; 2; 9; 10 chứa các đỉnh của các đường dẫn trên).

Với các đường đi trên, vòng lặp `while` của hàm `average` chỉ được thực hiện tối đa một lần lặp nên chúng ta rất khó để phát hiện các lỗi tiềm ẩn (có thể có) bên trong vòng lặp này. Các lỗi này có thể xảy ra khi vòng lặp này được thực hiện nhiều lần lặp. Ví dụ trên đã chỉ ra những hạn chế của phương pháp kiểm thử dòng điều khiển khi áp dụng cho các chương trình/đơn vị chương trình có chứa vòng lặp. Để giải quyết vấn đề này, chúng ta cần sinh thêm bảy ca kiểm thử ứng với bảy trường hợp sau:

1. Vòng lặp thực hiện 0 lần
2. Vòng lặp thực hiện 1 lần
3. Vòng lặp thực hiện 2 lần
4. Vòng lặp thực hiện k lần, $2 < k < n - 1$, với n là số lần lặp tối đa của vòng lặp
5. Vòng lặp thực hiện $n - 1$ lần
6. Vòng lặp thực hiện n lần
7. Vòng lặp thực hiện $n + 1$ lần

Chú ý rằng trong một số trường hợp chúng ta có thể không xác định được số lần lặp tối đa của các vòng lặp. Trong trường hợp này, chúng ta chỉ cần sinh bốn ca kiểm thử đầu tiên. Tương tự, trong một số các trường hợp khác, chúng ta không thể sinh ca kiểm thử để vòng lặp thực hiện $n + 1$ lần (trường hợp thứ 7). Khi đó, chúng ta chỉ cần sinh sáu ca kiểm thử còn lại (các trường hợp từ 1–6). Ví dụ, với vòng lặp `while` trong hàm `average` như Hình 7.4, vòng lặp này chỉ thực hiện lặp tối đa 100 lần nên chúng ta không thể sinh ca kiểm thử để nó thực hiện $n + 1 = 101$ lần. Kết quả là chúng ta chỉ cần sinh sáu ca kiểm thử đầu tiên như trong Bảng 7.5 nhằm kiểm thử vòng lặp này.

Bảng 7.5: Các ca kiểm thử cho kiểm thử vòng lặp `while` của hàm `average`

ID	Số lần lặp	Inputs	EO	RO	Note
tcl0	0	[-999, ...], 1, 2	-999		
tcl1	1	[1,-999], 1, 2	1		
tcl2	2	[1,2,-999], 1, 2	1.5		
tcl _k	5	[1,2,3,4,5,-999], 1, 10	3		
tcl _(n-1)	99	[1,2,...,99,-999], 1, 100	50		
tcl _n	100	[1,2,...,100], 1, 2	50.5		
tcl _(n+1)					

Với các chương trình/đơn vị chương trình có các vòng lặp liên kế, chúng ta tiến hành kiểm thử tuần tự từ trên xuống. Mỗi vòng lặp được kiểm thử bằng bảy ca kiểm thử như vòng lặp đơn giản (như đã mô tả ở trên). Trong trường hợp các vòng lặp lồng nhau, chúng ta tiến hành kiểm thử tuần tự các vòng lặp theo thứ tự từ trong ra ngoài (mỗi vòng lặp cũng dùng bảy ca kiểm thử như đã mô tả ở trên).

7.2 Tổng kết

Kiểm thử dòng điều khiển là một trong những phương pháp kiểm thử quan trọng nhất của chiến lược kiểm thử hộp trắng cho các chương trình/đơn vị chương trình. Phương pháp này cho phép phát hiện ra các lỗi (có thể có) tiềm ẩn bên trong chương trình/đơn vị chương trình bằng cách kiểm thử các đường đi của nó tương ứng với các dòng điều khiển có thể có. Để áp dụng phương pháp này, chúng ta cần xác định độ đo kiểm thử (cần kiểm thử với độ đo nào?). Tiếp theo, đồ thị dòng điều khiển của chương trình/đơn vị chương trình ứng với độ đo kiểm thử sẽ được tạo ra. Dựa vào đồ thị này, chúng ta sẽ sinh ra các đường đi độc lập. Số đường đi độc lập này là các trường hợp tối thiểu nhất để đảm bảo 100% độ bao phủ ứng với độ đo yêu cầu. Với mỗi đường đi, chúng ta sẽ sinh ra một ca kiểm thử sao cho khi nó được dùng để kiểm thử thì đường đi này được thực thi. Việc sinh các đầu vào cho các ca kiểm thử này là một bài toán thú vị. Chúng ta sẽ chọn một bộ đầu vào sao cho thỏa mãn các điểm quyết định có trong đường đi tương ứng. Giá trị đầu ra mong muốn ứng với mỗi bộ đầu vào của mỗi ca kiểm thử cũng sẽ được tính toán. Đây là bài toán khó và thường chỉ có các chuyên gia phân tích chương trình mới có thể trả lời chính xác giá trị này. Cuối cùng, các ca kiểm thử được chạy nhằm phát hiện ra các lỗi của chương trình/đơn vị chương trình cần kiểm thử. Khi một lỗi được phát hiện bởi một ca kiểm thử nào đó, nó sẽ được thông báo tới lập trình viên tương ứng. Lập trình viên sẽ tiến hành sửa lỗi (phát hiện vị trí của lỗi ở câu lệnh nào và sửa nó). Trong trường hợp này, chúng ta không chỉ thực hiện lại ca kiểm thử phát hiện ra lỗi này mà phải thực hiện lại tất cả các ca kiểm thử của đơn vị chương trình. Lý do chúng ta phải thực hiện công việc này là vì khi sửa lỗi này có thể gây ra một số lỗi khác.

Việc áp dụng phương pháp kiểm thử dòng điều khiển là khó và tốn kém hơn các phương pháp kiểm thử hộp đen (phân hoạch tương đương, phân tích giá trị biên, bảng quyết định, ...). Để áp dụng kỹ thuật này, chúng ta cần đội ngũ nhân lực về kiểm thử có kiến thức và kỹ năng tốt. Hơn nữa, chúng ta cần một sự đầu tư lớn về các nguồn lực

khác (tài chính, thời gian, ...) mới có thể thực hiện tốt phương pháp này. Đây là một yêu cầu khó và không nhiều công ty phần mềm đáp ứng được. Kiểm thử dòng điều khiển tự động hứa hẹn sẽ là một giải pháp tốt nhằm giúp cho các công ty giải quyết những khó khăn này. Hiện nay, đã có nhiều công cụ hỗ trợ một phần hoặc hoàn toàn các bước trong phương pháp này. Một môn trường lập trình mới nơi mà các công cụ lập trình trực quan được tích hợp với công cụ kiểm thử tự động nhằm giải quyết những khó khăn nêu trên. Eclipse1 là một ví dụ điển hình về xu hướng này. Trong cá phiên bản hiện nay của môi trường này đã được tích hợp công cụ kiểm thử tự động có tên là JUnit2. Công cụ kiểm thử này chưa hỗ trợ việc sinh các ca kiểm thử tự động nhưng nó cho phép chúng ta viết các kịch bản kiểm thử độc lập với mã nguồn và thực thi chúng một cách tự động trên một môi trường thống nhất.

Câu hỏi cuối bài:

1. Trình bày độ đo C1? Cho ví dụ?
2. Trình bày độ đo C2? Cho ví dụ?
3. Trình bày độ đo C3? Cho ví dụ?
4. Trình bày độ đo vòng lặp? Cho ví dụ?
5. Các bài tập thực hành:

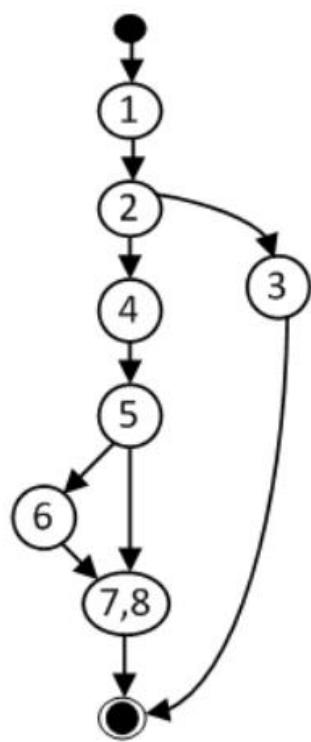
Bài 1. Cho đoạn chương trình sau:

```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.      return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.      x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```

1. Vẽ đồ thị dòng điều khiển cho đoạn chương trình trên
2. Tính hằng số cyclomatic (kí hiệu C) cho sơ đồ vừa vẽ?
3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3

Hướng dẫn:

1. Vẽ đồ thị:



2. Tính $C = P + 1 = 3$

3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3

Độ đo C1:

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5; 6; 7,8	2, 2, 3, 5	1		
tc2	1; 2; 3	0, 3, 2, 7	0		

Độ đo C2:

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5; 6; 7,8	2, 2, 3, 5	1		
tc2	1; 2; 3	0, 3, 2, 7	0		
tc3	1; 2; 4; 5; 7,8	2,3,4,5	lỗi chia cho 0		

Độ đo 3:

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5c1; 6; 7,8	0, 2, 3, 5	0		
tc2	1; 2; 4; 5c1; 5c2; 6; 7,8	2, 2, 2, 7	1		
tc3	1; 2; 4; 5c1; 5c2; 7,8	2,3,4,5	lỗi chia cho 0		
tc4	1; 2; 3	2,3,4,4	1		

Bài 2. Cho đoạn chương trình sau:

```
double    Power(double x, int n)
{
    double    result = 1;
    for(int i=1; i< n; i++)
        result *= x;

    return result;
}
```

1. Vẽ đồ thị dòng điều khiển cho đoạn chương trình trên
2. Tính hằng số cyclomatic (kí hiệu C) cho sơ đồ vừa vẽ?
3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3

Bài 3. Cho đoạn chương trình sau:

```
int sum(int a[], int n){
    int i;
    int total = 0;
    for(i=0; i<n; i++)
        total = total + a[i];
    return total;
}
```

1. Vẽ đồ thị dòng điều khiển cho đoạn chương trình trên
2. Tính hằng số cyclomatic (kí hiệu C) cho sơ đồ vừa vẽ?
3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3
4. Sinh các đường đi và các testcase đảm bảo độ đo cho vòng lặp

Bài 4: Cho mã nguồn của hàm Phân tích điểm số sinh viên theo nhóm như sau:

```
void analyze_scores(int scores[], int size) {
    int pass_count = 0, fail_count = 0;
    for (int i = 0; i < size; i++) {
        if (scores[i] >= 5) pass_count++;
        else fail_count++;
    }
    if (pass_count > fail_count) printf("Majority Passed\n");
    else printf("Majority Failed\n");
}
```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển
2. Tính độ phức tạp của đồ thị

3. Sinh testcase đảm bảo độ đo C2 và độ đo vòng lặp

Bài 5: Cho mã nguồn của hàm Tìm số lớn nhất như sau:

```
int find_max(int a, int b, int c) {  
    if (a > b) {  
        if (a > c) return a;  
        else return c;  
    } else {  
        if (b > c) return b;  
        else return c;  
    }  
}
```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển
2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C1 và C2

Bài 6: Cho mã nguồn của hàm Xử lý giá trị đầu vào như sau:

```
int process_input(int x) {  
    if (x < 0) return -1;  
    else if (x == 0) return 0;  
    else if (x < 10) return x * 2;  
    else return x * 3;  
}
```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển
2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C1 và C2

Bài 7: Cho mã nguồn của hàm Xác định số nguyên tố như sau:

```
bool is_prime(int n) {  
    if (n <= 1) return false;  
    for (int i = 2; i * i <= n; i++) {
```

```

        if (n % i == 0) return false;
    }
    return true;
}

```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển
2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C2 và độ đo vòng lặp

Bài 8: Cho mã nguồn của hàm Tìm phần tử lớn nhất trong mảng với điều kiện như sau:

```

int find_max_with_condition(int arr[], int size) {
    int max = -1;
    for (int i = 0; i < size; i++) {
        if (arr[i] % 2 == 0 && arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển đảm bảo độ đo C2 và C3
2. Tính độ phức tạp của mỗi đồ thị
3. Sinh testcase đảm bảo độ đo C2 và C3